

01. 개요

1. Mail 아키텍처 리팩토링 이유

- [알고리즘] : MailSide와 UDA0000 의 상호 호출 및 여러곳에서 데이터를 받고 있어 **코드 흐름의 부재**
 - [문서화 부족] : 여러 기능이 하나의 큰 컴포넌트에 집중되어 있어 **기능의 역할이 명확하지 않음**
 - [상태 초기화의 복잡성] : state변수 할당이 많아 **코드의 가독성이 부족**
 - [상태 업데이트의 비일관성] : 상태 업데이트가 여러 곳에서 이루어져 **코드의 흐름을 추적하기 어려움**
 - [비동기 처리의 불명확성] : 비동기 처리가 명확하지 않아 **콜백이 어디서 이루어지는지 알 수 없음**
 - [권한 체크 로직의 분산] : 권한 처리가 분산되어 있어 **흐름추적에 용이하지 않음**
 - [메뉴 관리 로직의 복잡성] : 동적메뉴 처리의 로직이 매우 복잡하여 **유지보수에 어려움이 있음**
 - [UI 구성 요소의 분리 부족] : UI가 명확하게 분리되어 있지 않고 **퍼블리싱 코드를 그대로 사용함**
 - [조건부 렌더링의 불명확성] : **렌더링 시점이 불명확함**
 - [불필요한 렌더링] : **지속적인 렌더링이 일어나고 있음**
 - [코드 스타일의 비일관성] : 여러명의 추가 개발로 인한 **코드 스타일이 달라 가독성이 떨어짐**

02. 리팩토링

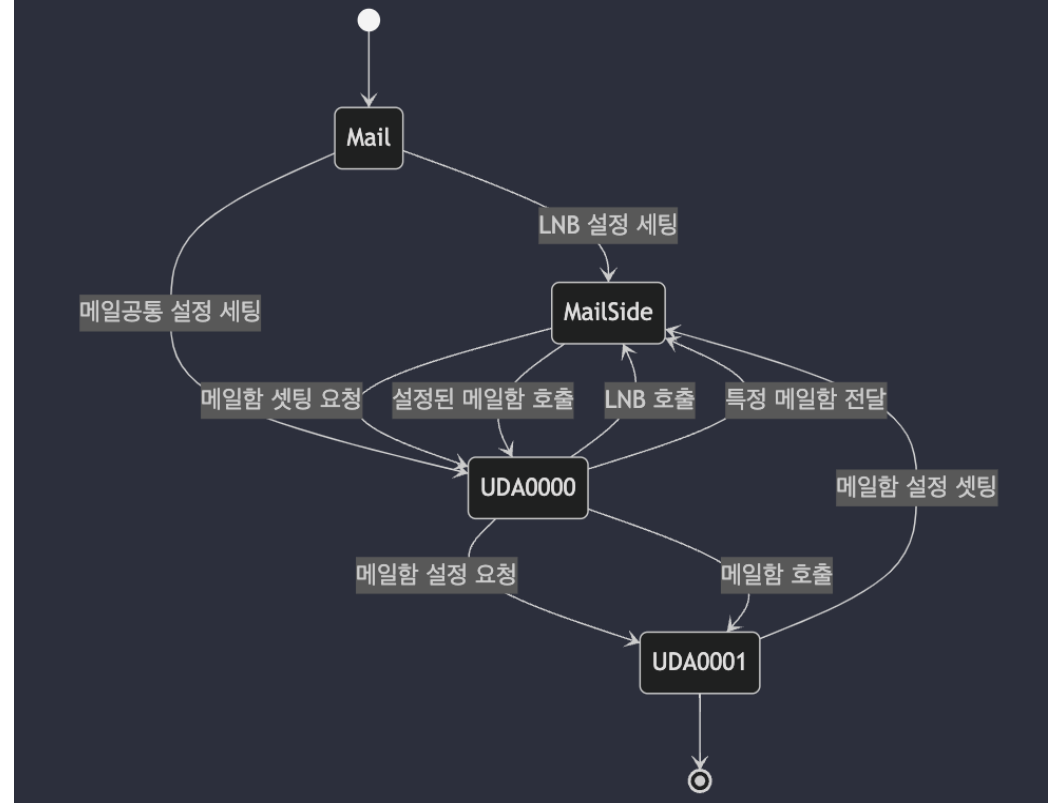
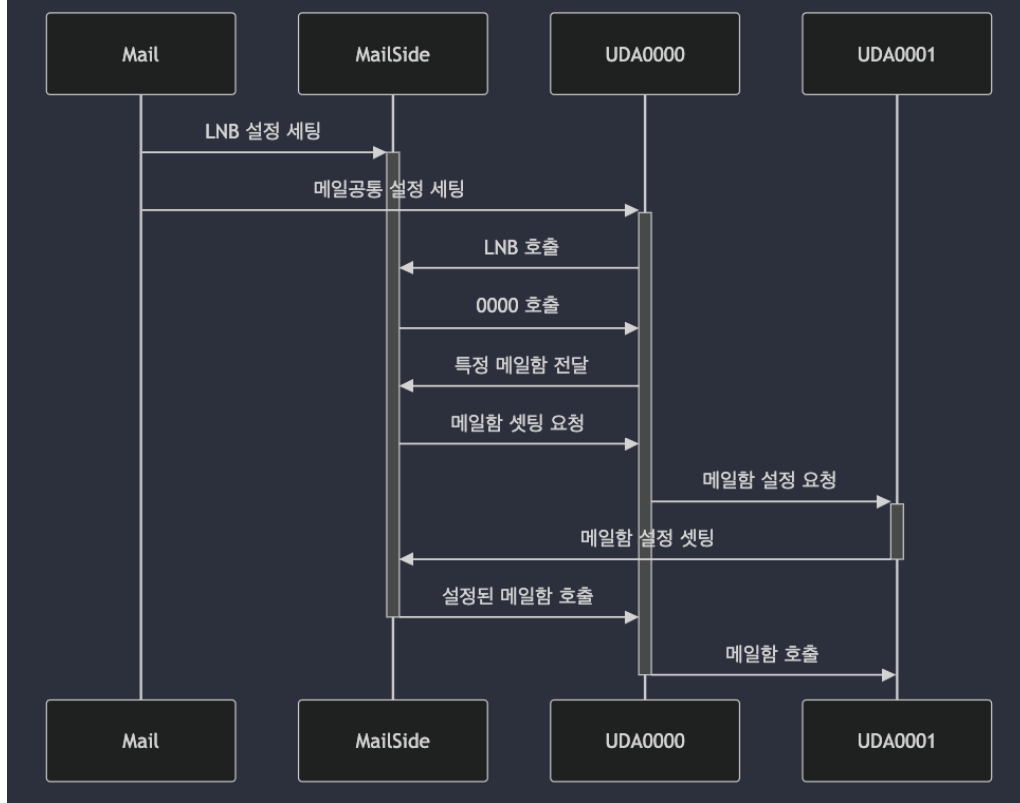
- 현재 메일 시스템은 **다수의 컴포넌트가 복잡하게 상호작용하며, 상태 관리와 코드 흐름이 명확하지 않아 유지보수성**이 낮습니다. 이에 따라 코드의 구조를 개선하고 가독성을 높이기 위해 리팩토링을 진행하고자 합니다.

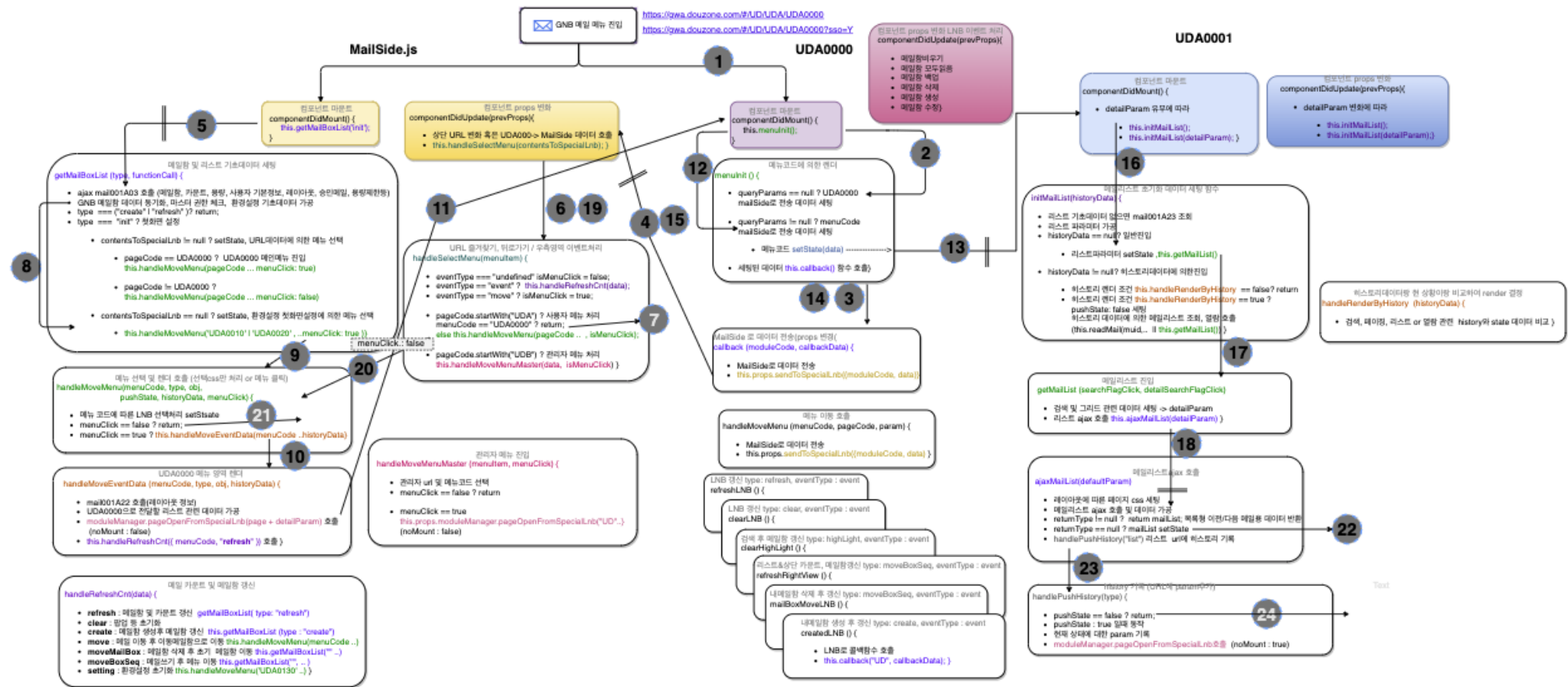
1.1 주요 문제점

1. 복잡한 코드 흐름

- 여러 **컴포넌트 간 양방향 데이터 전달**로 인해 코드 추적이 어려움
- 예: MailSide ↔ UDA0000(메일 코어) ↔ UDA0001(메일함) ↔ 메일함 내 연결컴포넌트(UDA00X, UDAP00X)

- 시퀀스 다이어그램, 상태 다이어그램





2. 상태 관리의 문제

- 상태 변수의 과도한 사용으로 코드 가독성이 저하됨
- setState 호출이 여러 곳에서 발생하여 상태 변경 추적이 어려움

//KLAGOP1-57562 메일리스트 render 최적화

```
const {dockPanel, boxName, conversionBoxName, searchText, dropdownlist, totalCount, totalUnseenCount, unseenTotal,
countColor, filterFlag, detailOpenYN, subjectCheckBox, searchFlag, detailSearchFlag, //검색영역
search_from, search_to, search_contentType, search_content, popupMailBoxOpen, search_boxName, searchOpenLoading,
search_mailBox, pageCode, search_term, search_attachYN, //디테일 검색영역
disabledYN, limitDisabledYN, viewDetail, allMailListType, seen, allChk, muid, mboxSeq, rejectFromEmail,
maillist, clickMail, layerType, viewYN, spamType, popupMoveOpen, topFncFlag, //상단 버튼 영역
dataType, viewHorizontalYN, showType, nowPage, pageSize, inputFlag, gridSizeChk, accountType, //그리드
pagingList, page, totalPage, //페이징
viewExpendYN, prevContent, nextContent, mailContent, contentDisplay, conFlagged, reparsingFlag, eachtrans, approvalMailSeq, approvalStatus, //상세
openLoading, isShowForUAAP012, p_comSeq, p_deptSeq, p_empSeq,
deleteClick, delLabelText, isShowAddrPop, isShowBoardPop, addrEmails, empSeq, delByFromClick, //팝업, 이소연, 4년 전 • Initial commit
keepOpen, keepUrl, keepSubject, moduleKey, //KEEP
exportOpen, orgLinkUrl, exportKeep, exportBoard, normalBoardName, exportLaw, //내보내기 KLAGOP1-58312
approvalPopupView,
isShowForUYAP004, addressUid, //통합주소록 프로필
useIntedAddr, //A10D-2317 연락처에 추가 or 주소록에 추가 분기로 사용
useAddressBookGw, //A10D-2940 주소록 사용여부 권한 적용
filterType, //[CSA10-11154_리팩토링] 필터 라벨 변경
sort, sortType, //[CSA10-11154_리팩토링] 정렬 라벨 변경
popupOneAIOpen, oneAIFileList, //A10D-2644 ONE AI 공통 컴포넌트 열람
} = this.state;
```

3. 문서화 부족

- 여러 기능이 하나의 큰 컴포넌트에 집중되어 있어 **기능의 역할이 명확하지 않음**

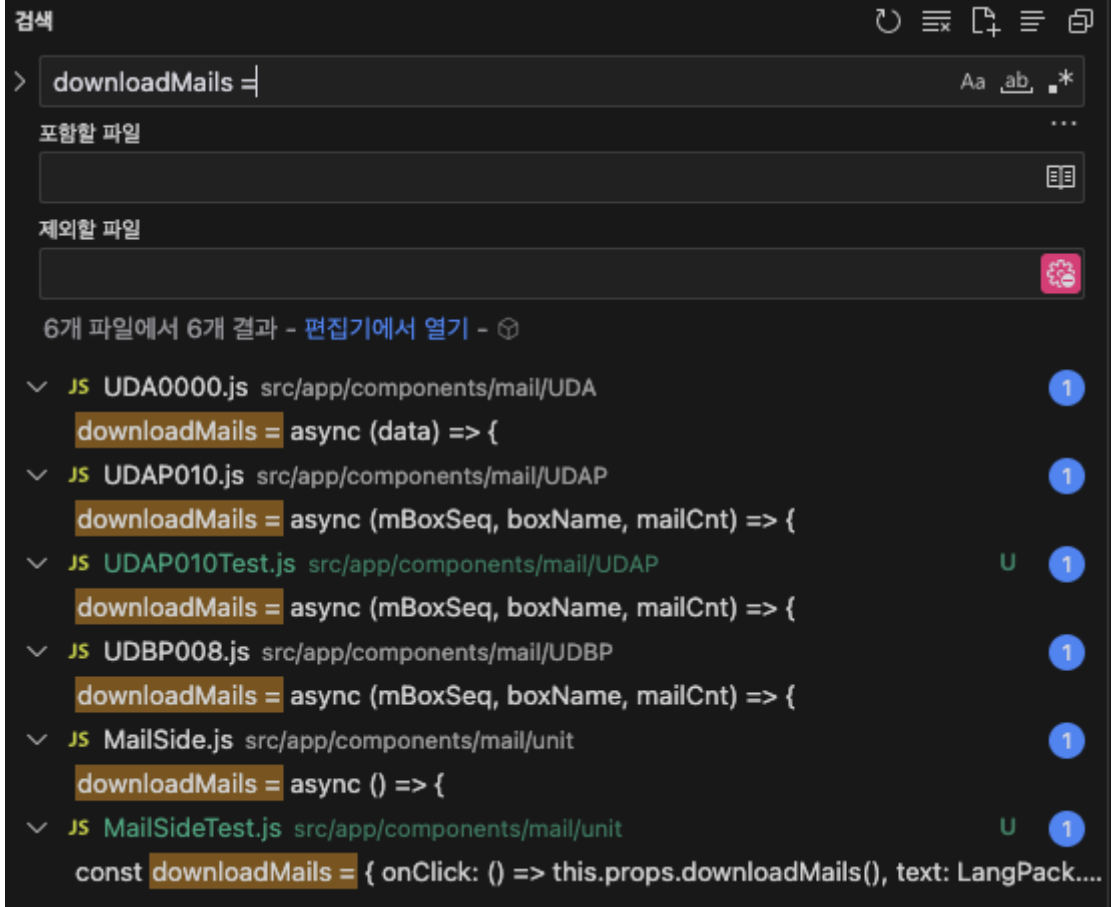
```

30 export default class UDA0001 extends React.Component {
4280   render() {
4574     <NewMailGridType01
4575       {...this.props}
4576       boxType={boxType}
4577       dataType={dataType}
4578       showType={showType}
4579       mailList={mailList}
4580       page={nowPage}
4581       boxName={boxName}
4582       allMailListType={allMailListType} // 안읽음/ 중요/ 첨부/ 받는사람
4583       handleCheckBoxGrid={this.handleCheckBoxGrid}
4584       handleViewDetail={this.handleViewDetail}
4585       handleSpamReport={this.handleSpamReport}
4586       handleCheckRemoveRow={this.handleCheckRemoveRow}
4587       handleNewFramePop={this.handleNewFramePop}
4588       handleMailWrite={this.handleMailWrite}
4589       handleDraftMailWrite={this.handleDraftMailWrite} // 임시보관함 메일 다시쓰기
4590       handleFlagged={this.handleFlagged}
4591       handleCheckSeenUnseen={this.handleCheckSeenUnseen} // 읽음 & 안읽음
4592       handleSendCancel={this.handleSendCancel}
4593       handleImmediatelySend={this.handleImmediatelySend}
4594       handleCancelSendMail={this.handleCancelSendMail} // 발송취소
4595       handleNonReception={this.handleNonReception} // 수신차단
4596       handleOpenProfile={this.handleOpenProfile} // 프로필 보기
4597       handleSearchMailFrom={this.handleSearchMailFrom} // KLAGOP1-57713 보낸사람으로 검색
4598       handleSearchMailTo={this.handleSearchMailTo} // KLAGOP1-57713 받은사람으로 검색
4599       handleDeleteByMailFromPop={this.handleDeleteByMailFromPop} // KLAGOP1-57713 이 사람이 보낸 메일 모두 삭제
4600       handleAddAddrList={this.handleAddAddrList} // KLAGOP1-57479 주소록에 추가
4601       userInfo={userInfo}
4602       pageContainer={this.pageContainer}
4603       listNum={pageSize}
4604       totalCount={totalCount}
4605       subjectCheckBox={subjectCheckBox} //제목만 보기 체크 값
4606       inputFlag={inputFlag}
4607       searchFlag={searchFlag} // 검색
4608       detailSearchFlag={detailSearchFlag} // 상세검색
4609       accountType={accountType} // 계정 노출 타입
4610       spamType={spamType} // 스팸메일함 연동 여부(값 있을 경우 연동 -> EWAL, JIRAN)
4611       handleApprovalMailPop={this.handleApprovalMailPop}
4612       useIntedAdddr={useIntedAdddr} // A10D-2317 메일 리스트 > 계정 클릭 > 연락처에 추가 or 주소록에 추가 분기로 사용
4613       useAddressBookGw={useAddressBookGw} // A10D-2940 메일 리스트 > 계정 클릭 > 연락처에 추가 or 주소록에 추가 주소록 사용여부 권한 적용
4614     />
4615   </div>
4616 }
4617 { /* 좌우분할 시 그리드 */ }

```

- 동일한 기능이 페이지별 1:1 매핑되어 반복적으로 구현됨
- 예: fnPrint() [인쇄] 함수가 여러 페이지에서 중복 사용됨

- 동일한 기능이 페이지별 1:1 매핑되어 반복적으로 구현됨
- 예: fnPrint() [인쇄] 함수가 여러 페이지에서 중복 사용됨



5. 비동기 처리 및 권한 체크 로직 분산

- 비동기 처리가 명확하지 않아 콜백 실행 위치가 불명확함
- 권한 체크 로직이 여러 곳에 분산되어 있어 유지보수가 어려움

6. UI 구성 및 조건부 렌더링 문제

- UI 요소들이 명확하게 분리되지 않음
- 퍼블리싱 코드가 직접 사용되어 가독성이 낮음

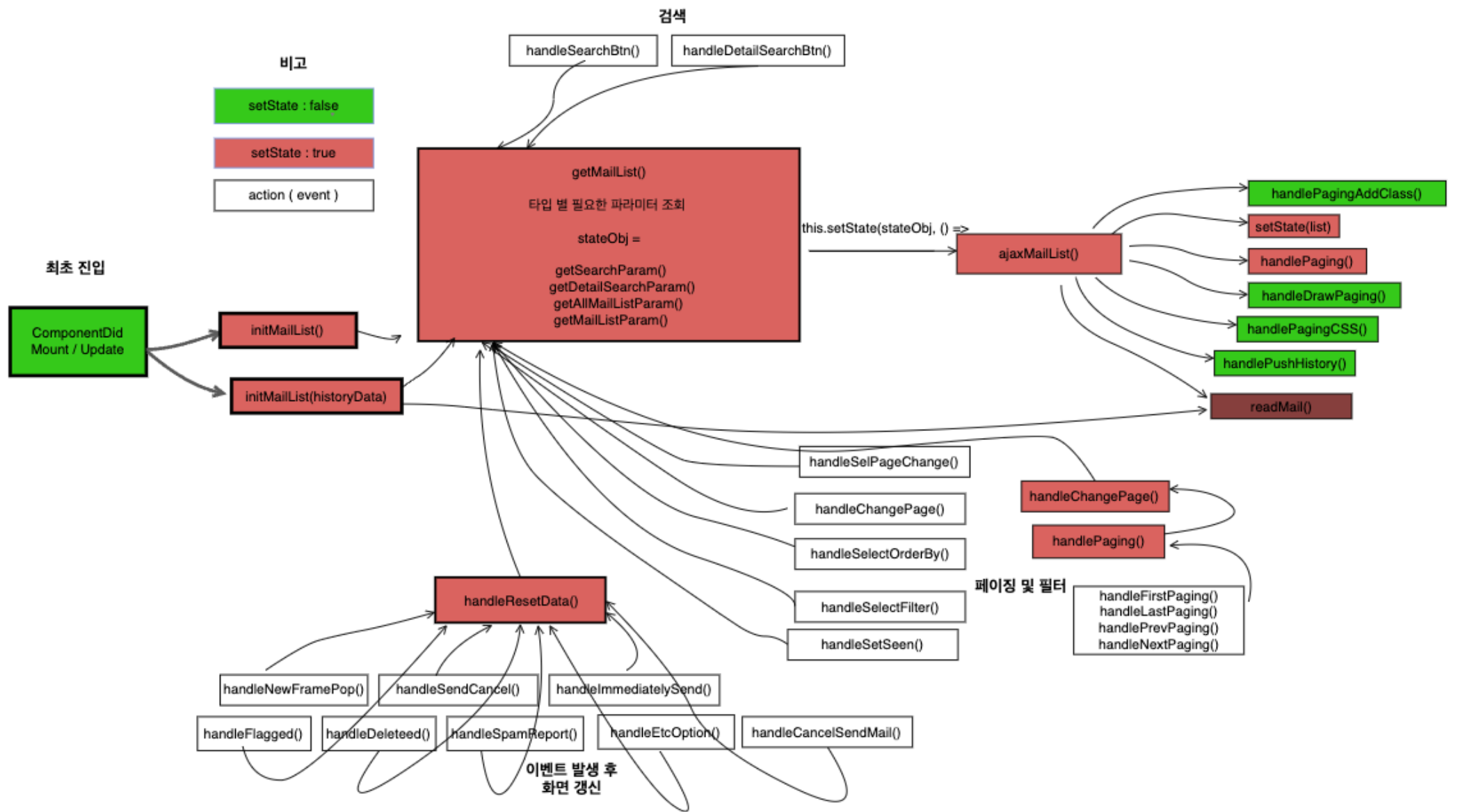
7. 불필요한 렌더링 발생

- 지속적인 렌더링으로 인해 성능 저하 발생 가능성 존재

8. 코드 스타일 불일치

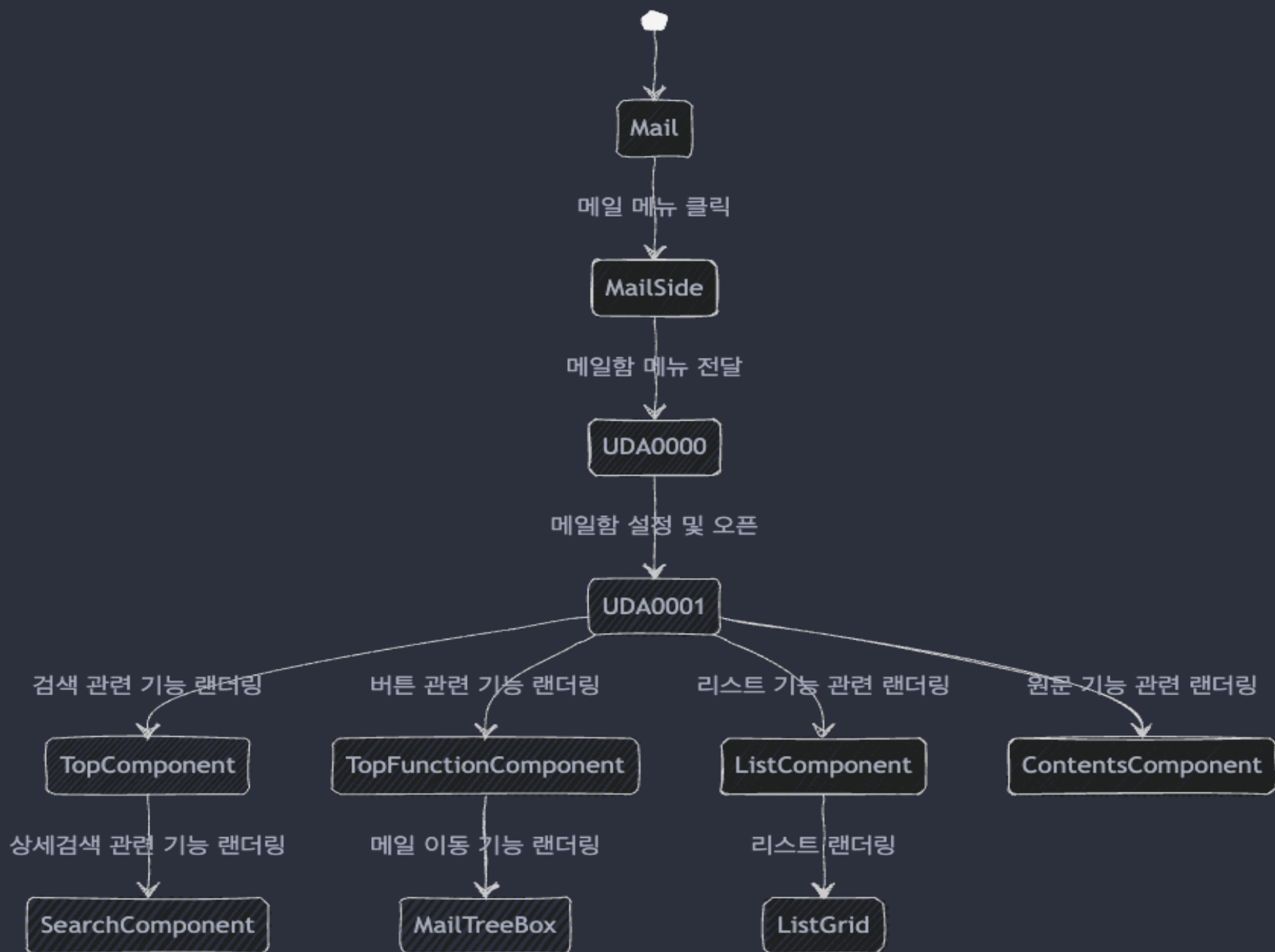
- 여러 개발자가 작업하면서 코드 스타일이 일관되지 않음

* 현재 메일 메뉴 선택 및 List 데이터 호출 시 **최소 6개의 이상의 페이지에 20회 이상의 함수 호출** 및 렌더링이 발생



2. 리팩토링 방안

2.1 각 페이지별 역할 분리



컴포넌트	역할
MailSide	LNB 메뉴 및 메뉴 카운트 기능
UDA0000 (메일 코어)	페이지 이동 기능
UDA0001 (메일함)	메일함 관련 기능

TopComponent	메일 검색 및 주요 기능 버튼
ListComponent	메일 리스트 및 기능 제공
ContentsComponent	메일 상세 및 기능
MailUtil	메일 공통 기능 (메일함 비우기, 읽음/안읽음 등)

2.2 메일 리스트 싱글톤 처리

- UDA0001 (메일함)에서 메일 리스트를 관리하고, 하위 컴포넌트 (TopComponent, ListComponent, ContentsComponent)에 **단방향으로 상태를 전달 (props)**하여 일관된 데이터 흐름 유지
 - UDA0001 → TopComponent
 - UDA0001 → ListComponent
 - UDA0001 → ContentsComponent

2.3 기능 공통화

- **중복되는 기능을 MailUtil 에서 공통 처리**하여 코드 재사용성을 높임
- 예: fnPrint() 기능을 중앙화하여 여러 페이지에서 호출 가능하게 개선

2.4 상태 관리 개선

- **상태 업데이트를 중앙화**하여 데이터 흐름을 명확히 정의
- setState 호출을 최소화하여 상태 변경 추적을 용이하게 함

2.5 UI 구성 요소 분리

- UI와 로직을 분리하여 코드의 가독성을 향상
- 퍼블리싱 코드와 JSX 코드를 분리하여 유지보수성을 개선

3. 기대 효과

항목	기대 효과
코드 가독성	코드 구조가 명확해져 유지보수 용이
상태 관리	일관된 상태 관리로 데이터 변경 추적 가능
코드 재사용성	기능 공통화를 통해 중복 코드 제거
협업 효율성	코드 스타일 통일 및 유지보수 용이

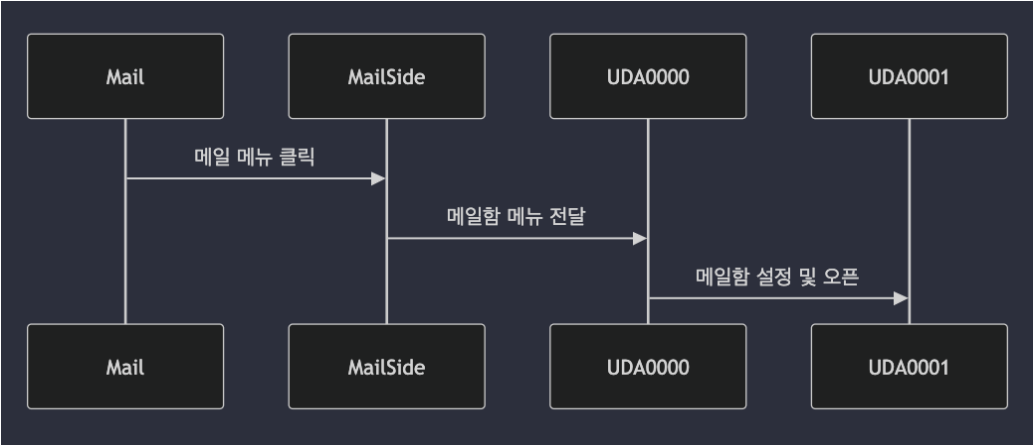
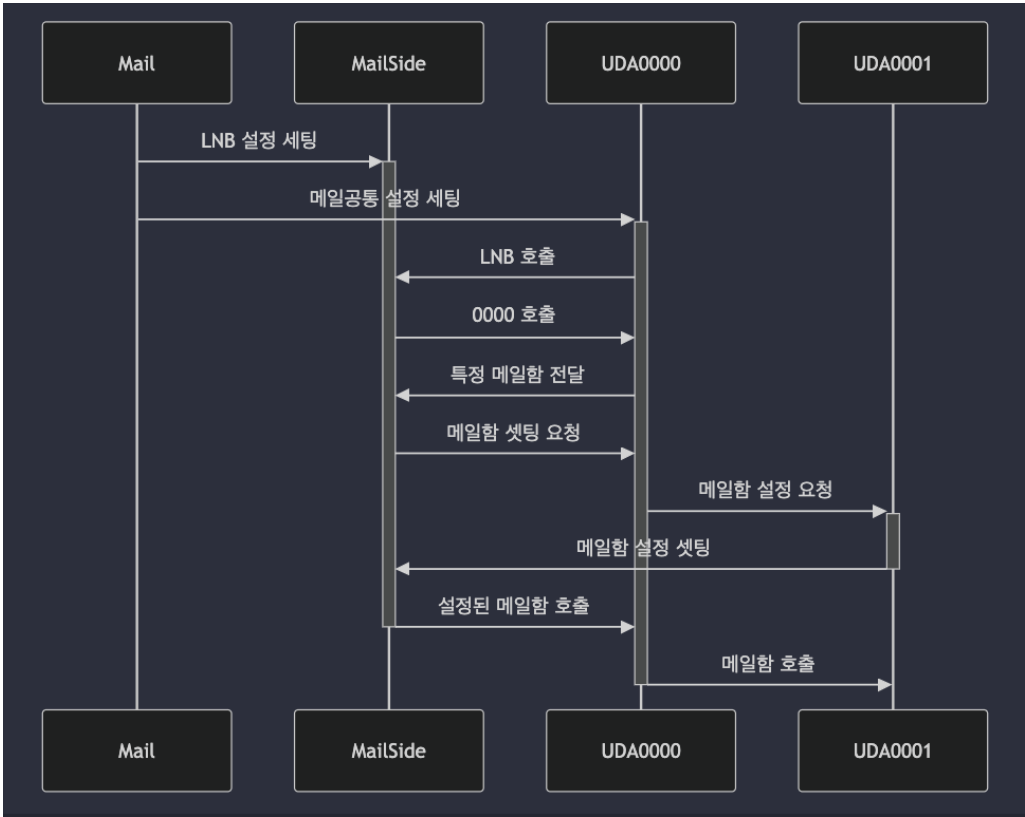
성능 개선	불필요한 렌더링 제거로 성능 최적화

4. 결론

이번 리팩토링을 통해 메일 시스템의 복잡성을 줄이고 유지보수성을 높이는 동시에, 성능 개선 및 협업 효율성을 높일 수 있습니다. 이를 통해 향후 기능 확장이 용이한 구조를 확보할 수 있을 것입니다.

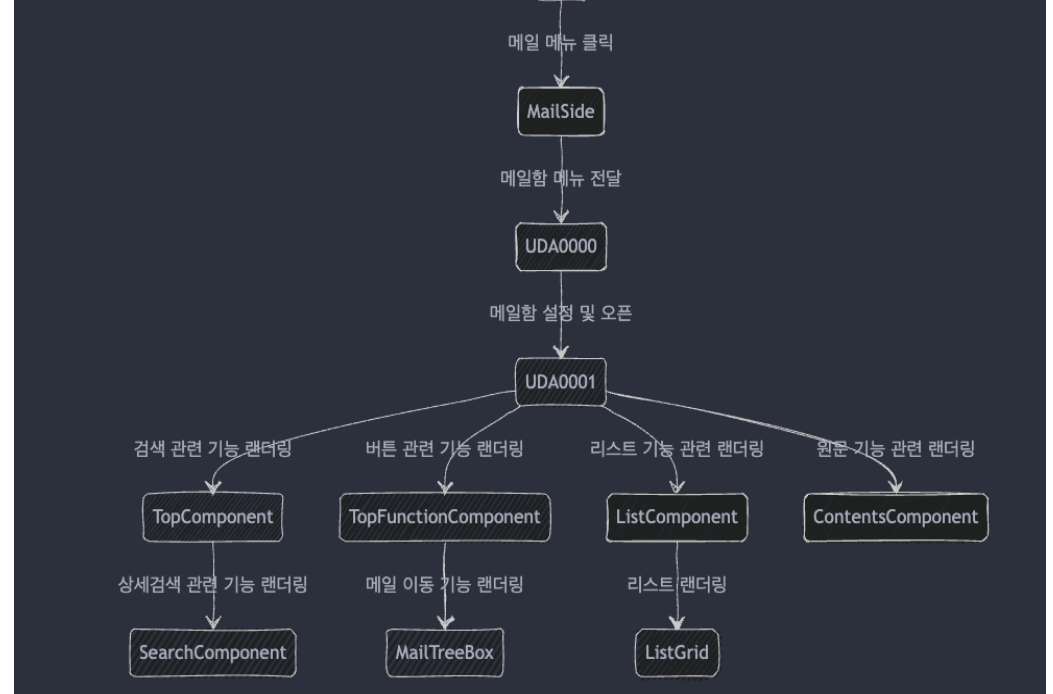
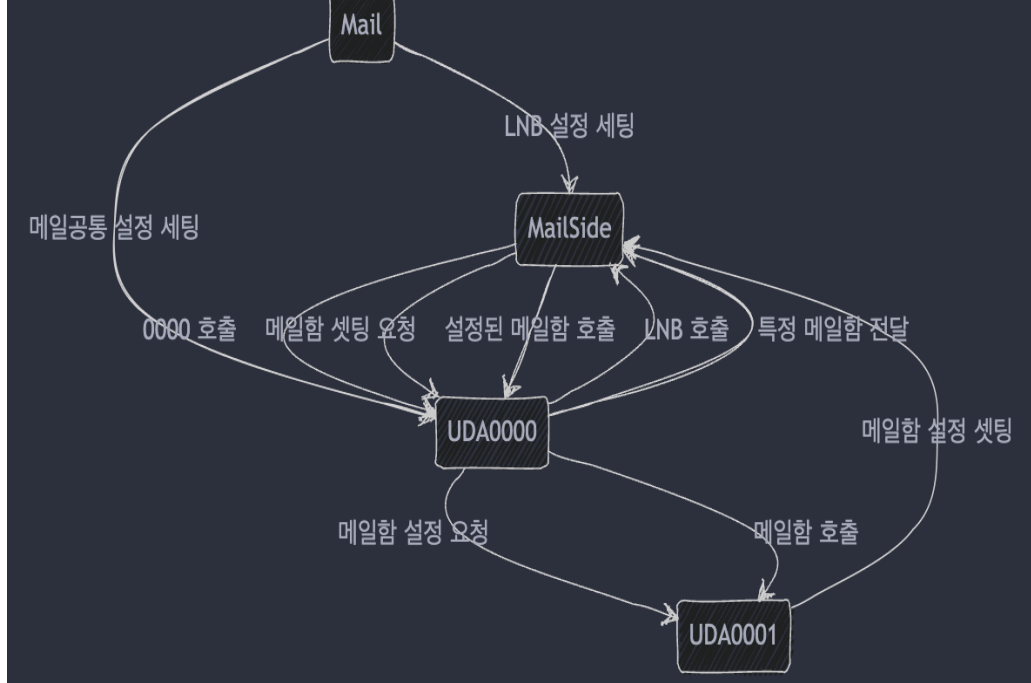
03. 리팩토링 전, 후 코드 비교 (wiki)

1. 시퀀스 다이어그램

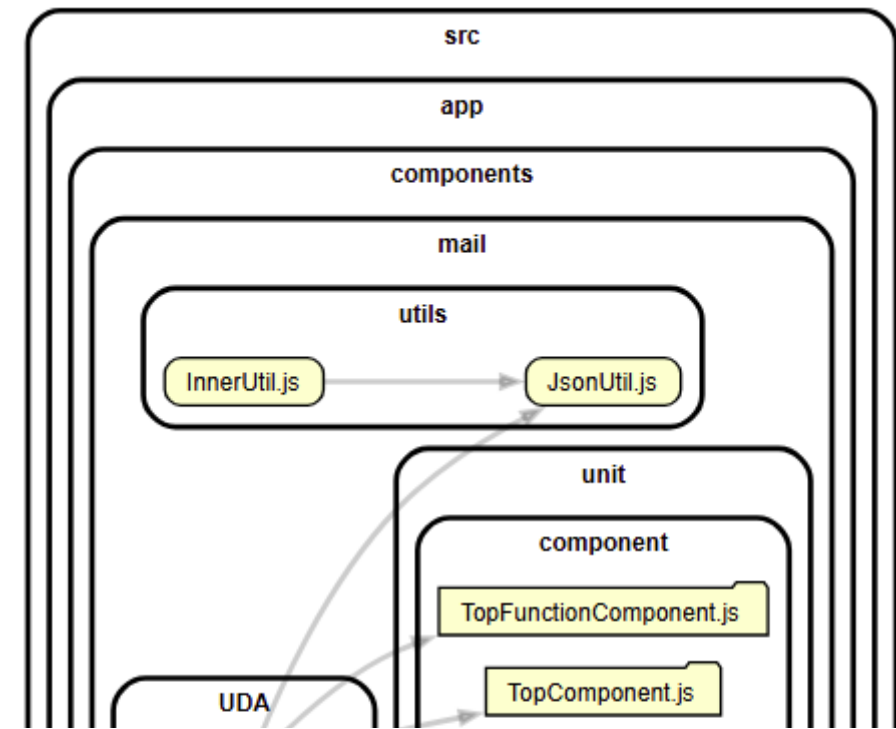
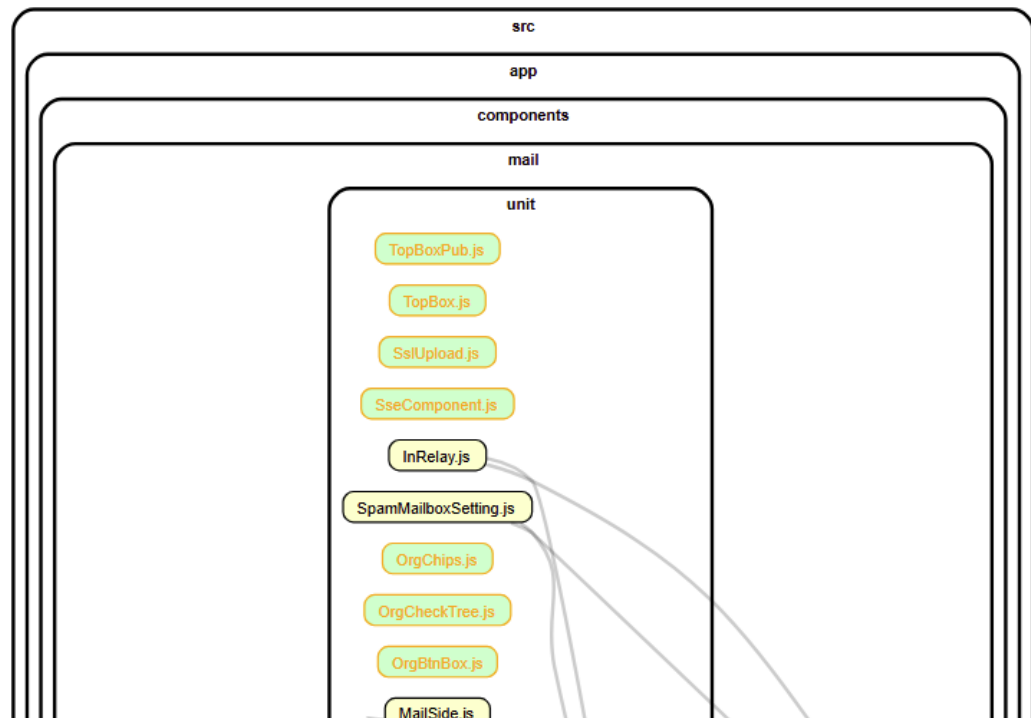


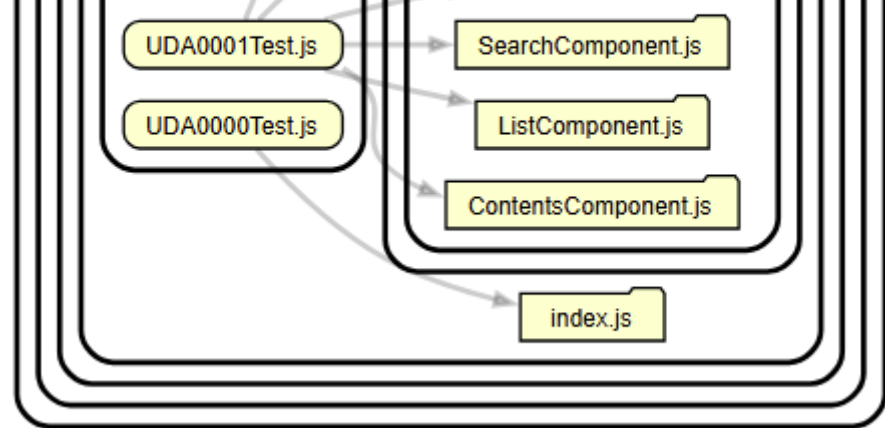
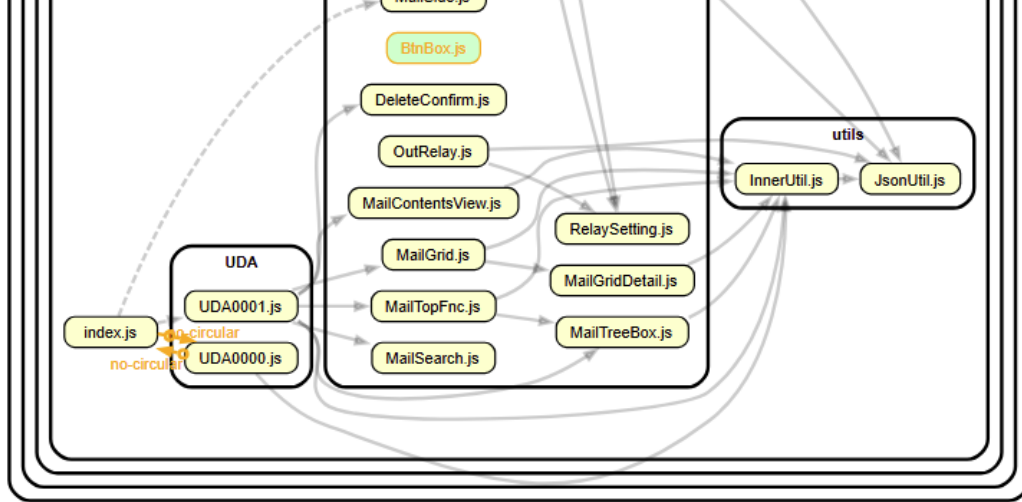
2. State 다이어그램





3. 의존성





4. 컴포넌트 사이즈 및 간략화

- MailSide - 1996줄 -> 699줄, **약 1300줄 감소**

JS MailSide.js ×

src > app > components > mail > unit > JS MailSide.js > ...

```

10  export default class MailSide extends Compon
1582  render() {
1991      </>
1992  }
1993  </div>
1994  );
1995  }
1996  }

```

JS MailSideTest.js U ×

src > app > components > mail > unit > JS MailSideTest.js > ...

```

14  export default class MailSide extends Compon
552  render() {
553      ...
694  }
695  </div>
696  );
697  };
698  }
699

```

- Mail0000 - 822줄 -> 177줄, **약 650줄 감소**

JS UDA0000.js ×

src > app > components > mail > UDA > JS UDA0000.js > ...

```

11  export default class MainPage extends React.
507  render() {
818      </>
819

```

JS UDA0000Test.js U ×

src > app > components > mail > UDA > JS UDA0000Test.js > ...

```

5  export default class MainPage extends React.
78  render() {
173      </>
174

```

```

819      }
820    }
821  }
822 }

174      }
175    }
176  }
177 }

```

- Mail0001 - 5104줄 -> 449줄, 약 4650줄 감소

JS UDA0001.js 1, ↓M, M ×

src > app > components > mail > UDA > JS UDA0001.js > ...

```

30  export default class UDA0001 extends React.C
4280    render() {
5102      );
5103    }
5104  }

```

JS UDA0001Test.js U ×

src > app > components > mail > UDA > JS UDA0001Test.js > ...

```

28  export default class UDA0001 extends React.C
402    render() {
447      );
448    }
449  }

```

5. 코어 개발 현황

메일 Component 개발	ListComp onent 개 발	박재영	• ListComponent 개발 ◦ 기본메일함 개발 ▪ 동적메일함 구조 개발 ◦ 수신확인함 개발 ◦ 레이아웃 개발 ▪ 좌우분할, 상하분할, 리스트 레이아웃 기능 개발 ◦ 페이징 개발
ContentsComponent 개발	메일 상세 개발	박재영	◦ 새창열기 기능 개발 ▪ FuncUtil (공통함수) 추가 ◦ 이전메일, 다음메일 기능 ▪ 조회된 리스트 외 가지고 있는 메일 목록(다음, 이전메일)을 지속적으로 가져오도록 추가 ◦ 본문 조회 기능 ▪ 본문 조회시 결과 파라미터 단순화 ◦ 사용자 기능 개발 ▪ TopFunctionCompont 위임
메일 Side개발	퀵 메뉴 개 발	박재영	(04.09) • 메일 Lnb 내 퀵메뉴 개발 ◦ 이름바꾸기 ◦ 내 메일함 추가 ◦ 하위메일함 추가 ▪ 하위메일 추가시 포커싱 및 상위트리 오픈하도록 수정 (04.08) • 메일 Lnb 내 퀵메뉴 개발

			8. 복구기 ○ 이름바꾸기 ○ 백업
메일 Popup 개발	Popup 개발	박재영	• 메일 상세 Popup 개발 ○ 공통 호출을 위한 기능 공통화 작업 ○ 팝업 내 기능 구현 ■ 콜백 기능 구현
메일 코어 개발	메일 History 개발	박재영	• 메일 전용 History 개발 ○ 뒤로가기 및 앞으로가기 ■ 페이지 및 검색 조회 등 기록 유지 ■ 이전 검색 데이터 유지 ■ 메일 본문 오픈 유지 • HistoryStore 추가 ○ 스택에 상태값 추가 ○ 리스트 데이터 변경시 히스토리 상태 변경작업
메일 Component 개발	TopCompo nent 개발	박재영	• 카운트 기능 • 새로고침 기능 • 간편검색 기능 ○ 수신확인함 검색조건 기능 • 상세검색 기능 ○ 보낸사람, 받는사람, 내용, 메일함, 기간, 첨부파일 등 • 즐겨찾기 기능
	TopFuncti onCompon ent 개발		• 메일 버튼 기능 구현 ○ 읽음/안읽음 기능 ○ 삭제 기능 ○ 스팸신고 기능 ○ 수신차단 기능 ○ 답장/전체답장/전달/다시보내기 기능 ○ 인쇄 기능 ○ 내보내기 기능 ○ 메일이동 기능 ○ PC 저장 기능 ○ 중요표시 기능 ○ 주소록 추가 기능 • 메일 정렬 기능 구현 ○ 메일함 별 정렬 구현 • 메일 필터 기능 구현 ○ 메일함 별 필터 구현
	ListCompo nent 개발		• 기본메일함 개발 ○ 전체메일함, 보낸메일함, 받은메일함 등 • 수신확인함 개발 • 레이아웃 개발 ○ 좌우분할

		박재영	◦ 상하분할 ◦ 리스트 ▪ 본문 보기 ▪ 새창열기 기능 ▪ 미리보기 기능 ▪ 개별 사용자 기능 개발 ◦ 사용자별 메일쓰기 ◦ 보낸사람으로 검색 ◦ 받은사람으로 검색 ◦ 프로필 보기 ◦ 연락처추가 ◦ 수신차단 ◦ 이 사람이 보낸 메일 삭제 • 페이징 개발
	ContentsComponent 개발		• Contentst Component 개발 ◦ 새창열기 기능 개발 ◦ 이전메일, 다음메일 기능 ◦ 본문 조회 기능 ◦ 사용자 기능 개발 ◦ 메일쓰기, 검색, 차단 등
메일 코어 개발	UDA0001 (동적 메일함) 개발		• UDA0001 (동적메일함) 개발 ◦ 메일함 별 주요 정보 설정 추가 (mailbox) ▪ API, menuCode, boxSeq, boxName ◦ 메일함 별 환경설정 조회 기능 ◦ LNB 카운트 조회 및 갱신 기능 ◦ 사용자별 레이아웃 설정 추가 ▪ 상하분할 ▪ 좌우분할 ▪ 리스트 ◦ 상태값 변경 기능 추가 ▪ Mailbox (메일함 상태) ▪ MailList (메일리스트 상태) ◦ 읽음, 안읽음 기능 추가 ◦ 동적 랜더링 기능 추가 ▪ TopComponent ▪ TopFunctionComponent ▪ ListComponent ▪ ContentsComponent
	UDA0000 (메일 공통) 개발		• UDA0000 (메일공통) 개발 ◦ 각 페이지 이동 기능 개발 ◦ LNB 카운트 갱신 기능 개발 ◦ 즐겨찾기 기능 개발
	MailSide 개발		• MailSide 개발 ◦ t_co_menu로 전달 받은 menuItem 을 기준으로 MailSide 개발 ▪ 메일함과 및 하위메일함을 menuItem에 결합 ◦ 동적 메일함 랜더링 기능 ▪ 하위메일함 랜더링 기능 추가 ◦ 카운트 갱신 기능 추가 ◦ 사용자 권한 체크 기능 추가 ◦ 수발신 권한 체크 기능 추가

			- 메뉴 이동 기능 추가 - 하위메일함 생성 및 이동 기능 개발 - LNB 킷 버튼 메뉴 추가 (읽음, 비우기, 백업, 메일함추가 등) - 메뉴 클릭시 지속적으로 갱신 되도록 수정 - 하위 메뉴함의 하위메뉴 카운트 미노출 이슈 수정 - 모듈 탭 종료 후 다시 실행 시 오류 수정
검토	코드 검토	박재영	- 메일 FrontEnd 코드 검토 - MailSide (LNB 메뉴) ↔ UDA0000(메일 공통) ↔ UDA0001(메일함) ↔ 메일팝업, 메일기능, 메일검색 양방향으로 전달 - 각 페이지에서 setState를 하여 각각의 상태를 가지고 있어 실질적으로 데이터가 어떻게 변경되어있는지 확인 불가 - 방대한 양의 State 객체로 인해 파라미터가 어떻게 사용되고 미사용되는지 알 수 없음 - 페이지당 기능이 1:1 매핑이 되어있어 동일한 기능이 여러곳에 존재 - 퍼블리싱 코드를 그대로 사용함으로써 JSX 코드 및 기능의 가독성 저하 - 개선방안 - 각 페이지별 역할 부여 - MailSide : LNB의 기능 및 메뉴 카운트 기능 - UDA0000(메일코어) : 페이지 이동 기능 - UDA0001(메일함) : 메일함 기능 - TopComponent(메일 상단 기능) : 메일 검색, 주요 기능버튼 - ListComponent(메일 리스트) : 메일 리스트 및 기능 - 메일리스트 싱글톤 처리 - ContentsComponent(메일 상세) : 메일 상세 및 기능 - MailUtil(메일 공통 기능) : 메일함 비우기, 메일함 추가, 메일읽음, 안읽음 기능 등 - 메일함(UDA0001)에서 가져온 메일리스트를 하위 컴포넌트에 전달하여 필요한 상태에 따라 공통(UDA0001)에서 업데이트 처리(단방향) - UDA0001 → TopComponent - UDA0001 → ListComponent - UDA0001 → ContentsComponent - 기능 공통화 - 인쇄 기능과 같은 공통 기능은 중복을 제거하고, 하나의 함수로 재사용할 수 있도록 구조를 개선

6. 그 외 다수 개발